

Relatório Técnico: Implementação DevOps e Cloud-Native - Loja Veloz

Data: 30 de Maio de 2026 **Disciplina:** Práticas de DevOps e Cloud-Native **Projeto:** Plataforma de E-commerce Loja Veloz

1. Vídeo de Demonstração

Para uma visão prática da arquitetura e do funcionamento do ambiente desenvolvido, assista ao vídeo de apresentação do projeto no link abaixo:

Link do YouTube: <https://youtu.be/dfpseP9zb3o>

2. Introdução e Objetivos

Este relatório técnico documenta o processo de modernização e migração da infraestrutura da plataforma de e-commerce **Loja Veloz** para um ambiente Cloud-Native baseado em contêineres e Kubernetes. O objetivo principal do projeto acadêmico foi aplicar os conceitos de DevOps abordados durante a disciplina, estabelecendo um ambiente de alta disponibilidade, escalável e com integração e entrega contínuas (CI/CD).

A aplicação baseia-se em uma arquitetura de microsserviços, englobando os módulos de **Pedidos** e **Pagamentos**, orquestrados para operar em conjunto através de comunicação assíncrona.

3. Arquitetura da Solução

O sistema foi dividido de maneira modular para evitar ponto único de falha e garantir escalabilidade independente. Os componentes arquiteturais incluem:

- **Microsserviço de Pedidos (Node.js):** Responsável por recepcionar as intenções de compra dos clientes, persistindo-as em banco de dados e enviando eventos de criação de pedido para a fila de mensageria.
 - **Microsserviço de Pagamentos (Node.js):** Consome de maneira assíncrona os eventos gerados pelo serviço de pedidos, simulando a aprovação ou recusa do pagamento das transações financeiras.
 - **RabbitMQ (Message Broker):** Garante o desacoplamento e a comunicação assíncrona resiliente entre Pedidos e Pagamentos. Caso o serviço de Pagamentos esteja indisponível, as mensagens são retidas na fila até a normalização do serviço.
 - **PostgreSQL:** Banco de dados relacional escolhido para fornecer consistência (ACID) às informações transacionais críticas dos pedidos da plataforma.
-

4. Estratégia de Containerização (Docker)

Para garantir que a aplicação rode de forma idêntica e isolada desde o ambiente de desenvolvimento até a produção, todos os serviços foram empacotados em imagens Docker.

Otimização e Segurança dos Dockerfiles

A elaboração dos **Dockerfiles** seguiu boas práticas focadas em segurança e performance ("Production Readiness"):

- **Base Images Enxutas:** Utilização de imagens base baseadas em Alpine Linux (`node:alpine`), reduzindo drasticamente a superfície de ataque e o tempo de download/build.
 - **Multistage Build:** Implementação (quando aplicável) de builds em múltiplas etapas, garantindo que dependências de desenvolvimento e ferramentas de compilação não sejam levadas para a imagem final de produção.
 - **Execução como Usuário Não-Root:** Configuração de diretivas `USER` para evitar a execução da aplicação como administrador (root) dentro do contêiner, mitigando riscos de vulnerabilidades do tipo *privilege escalation*.
-

5. Ambiente de Desenvolvimento (Docker Compose)

O arquivo `docker-compose.yml` foi construído para unificar e facilitar a execução local do projeto. Com um único comando (`docker-compose up`), todo o ecossistema é provisionado, englobando:

- A rede virtual isolada (Bridge Network `loja-veloz-net`).
 - Os volumes nomeados (`pgdata`) para a persistência dos dados do banco de dados, evitando perda de informações no reinício dos contêineres.
 - O mapeamento correto das variáveis de ambiente e ordenamento de subida de serviços utilizando `depends_on` e `healthchecks` rigorosos, garantindo, por exemplo, que as APIs Node.js só iniciem após o banco e o RabbitMQ estarem prontos.
-

6. Integração e Entrega Contínuas (CI/CD)

A esteira de CI/CD foi automatizada através do **GitHub Actions** (`.github/workflows/ci-cd.yml`). Esta pipeline é acionada a cada alteração na branch principal (Main) ou via Pull Requests.

A pipeline executa as seguintes etapas (Jobs):

1. **Linting e Testes:** Validação da qualidade do código para evitar que código quebrado siga no fluxo.
2. **Build das Imagens Docker:** Compilação automatizada das imagens dos serviços de Pedidos e Pagamentos.
3. **Publicação (Push):** Envio seguro das imagens validadas para um Container Registry, tornando-as prontas para o deploy.

Esta prática garante um ciclo de feedback rápido, além de evitar implementações manuais propensas a erro.

7. Prontidão para Produção e Orquestração (Kubernetes)

O próximo passo lógico para a alta disponibilidade em Nuvem foi o mapeamento dos serviços para o **Kubernetes**. Dentro do diretório `/k8s`, encontram-se os manifestos declarativos da infraestrutura, incluindo:

- **Deployments:** Configurados para suportar réplicas múltiplas das APIs, garantindo que o tráfego possa ser balanceado e tolerante a quedas de pods individuais.

- **Services:** Provisão de IPs internos estáveis (ClusterIP) para a comunicação fluida entre os pods e a exposição controlada ao mundo externo.
- **ConfigMaps e Secrets:** Separação limpa entre configuração não-sensível e dados sensíveis (como senhas do Postgres e URIs do RabbitMQ), não deixando credenciais hard-coded no código ou imagens.

Justificativa de Decisões DevOps

Todas as decisões arquiteturais tomadas visaram um ambiente "Production Ready" (Pronto para Produção). A containerização padronizou as entregas; o RabbitMQ protegeu as transações de perdas via assincronicidade; e o Kubernetes junto do CI/CD garante o provisionamento escalável (Horizontal Pod Autoscaling - HPA, em visões futuras) com observabilidade aprimorada.

Conclusão O projeto cumpre com sucesso os requisitos propostos para o desafio de modernização de infraestrutura, consolidando um ambiente resiliente, seguro e completamente alinhado à filosofia DevOps e padrões Cloud-Native contemporâneos.